

OEE Digitalization with IoT

Real-Time TRS Monitoring with ESP32, MQTT and Node-RED

Author: Oumaima Yaakoubi

Program: Industrial Maintenance and Instrumentation (IMI), INSAT

Date: Mai, 2026

Abstract

This report presents a complete IoT pipeline for real-time OEE (TRS) monitoring in an industrial production line. An ESP32 node simulated in Wokwi publishes machine state over MQTT while a Node-RED flow computes Availability (TD), Performance (TP), Quality (TQ) and the global OEE in real time. An integrated alert system triggers visual, audio and remote notifications when OEE crosses critical thresholds. The system was validated by running the full chain from sensor to dashboard and confirming that alerts trigger correctly when OEE crosses the critical threshold.

1 Introduction: Why OEE Matters

OEE (Overall Equipment Effectiveness), known in French as TRS (Taux de Rendement Synthétique), is the universal indicator of production efficiency. It measures the ratio between what a machine could ideally produce and what it actually produced, factoring in all losses.

In most factories, OEE is still computed manually at the end of a shift. By the time the number is ready, the losses have already happened. Micro-stops under five minutes are never recorded. The data is unreliable, the analysis is late and continuous improvement is blocked.

Key Insight

Real-time OEE is not just a better report. It is a different way of managing production. Losses become visible while they happen, not after the shift.

1.1 The Three Components of OEE

The global OEE is computed as $OEE = TD \times TP \times TQ$. Performance thresholds follow the SEMI E79 standard: above 85% is world class, 65–85% is average, 45–65% is low and below 45% is critical.

2 Objective of This Project

The objective is to design and validate an IoT pipeline that measures the three components of OEE automatically and in real time using only open-source tools and low-cost hardware, without touching the machine's existing PLC.

- Build a complete acquisition-to-dashboard chain using an ESP32, MQTT and Node-RED.
- Validate the OEE computation algorithms in simulation before any industrial deployment.
- Trigger automatic alerts when OEE drops below the critical threshold.
- Document the path toward a fully non-invasive industrial deployment.

3 System Architecture

The system is composed of three clearly separated layers, each with a distinct responsibility.

3.1 Acquisition Layer

The acquisition layer is an ESP32 DevKit C V4 running a firmware written in Arduino C++. It reads three simulated sensors:

- A **potentiometer** on analog pin 34, which simulates the machine's current consumption (0 to 5000 mA) and rotation speed.
- An **ultrasonic sensor HC-SR04** on pins 23 (trigger) and 5 (echo), which counts good parts by detecting objects closer than 15 cm.
- A **push button** on pin 17, which the operator presses to declare a reject.

Two LEDs (green on pin 18, red on pin 19) and a buzzer (pin 25) provide local feedback. The firmware also performs the full OEE computation locally, including debounce logic, downtime tracking and threshold detection.

3.2 Communication Layer

The communication layer is MQTT over Wi-Fi. The ESP32 connects to the public broker `broker.mqttdashboard.com` on port 1883. Every two seconds it publishes a single JSON message to the topic `pfa/trs/data`. It also subscribes to `pfa/trs/commands` to receive live configuration updates from the dashboard: nominal RPM, planned production time and machine name. This two-way channel lets the operator reconfigure the system without reflashing the ESP32.

3.3 Processing and Visualization Layer

The processing layer is a Node-RED flow. It subscribes to `pfa/trs/data`, parses the JSON payload, re-computes TD, TP, TQ and the global OEE for consistency, stores each record in SQLite and updates a live dashboard. The dashboard includes gauges for the four indicators, a history graph of RPM, a CSV export, an HTML report generator and a PIN-protected access to historical exports.

A second path in the flow handles alerts. When the global OEE drops below 45%, Node-RED sends an email and a Telegram notification to the production manager, in parallel with the local buzzer and red LED triggered by the ESP32.

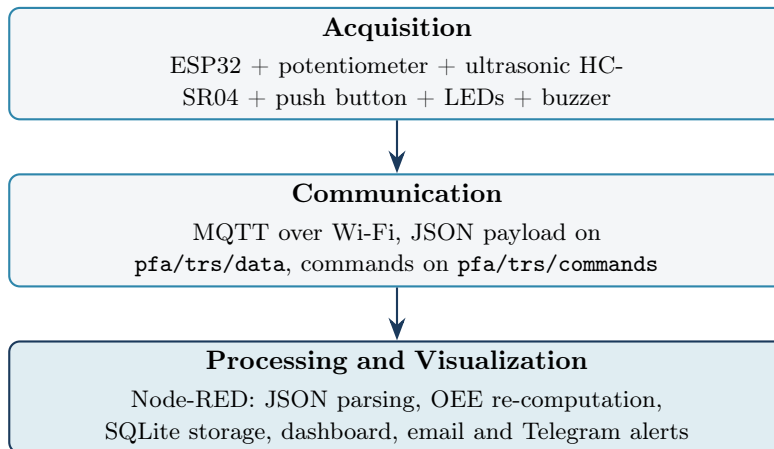


Figure 1: The three layers of the IoT OEE pipeline, from sensor acquisition to dashboard and alerts.

4 OEE Calculation

Each component is computed from raw sensor data.

Availability (TD)

$$TD = \frac{TR_{\text{planned}} - \text{downtime}}{TR_{\text{planned}}} \times 100$$

Measured from the current sensor. The machine is considered running when current exceeds 500 mA.

Performance (TP)

$$TP = \frac{RPM_{\text{actual}}}{RPM_{\text{nominal}}} \times 100$$

Measured from the potentiometer which simulates rotation speed.

Quality (TQ)

$$TQ = \frac{\text{goodCount}}{\text{goodCount} + \text{rejectCount}} \times 100$$

Good parts are counted by the ultrasonic sensor. Rejects are declared by the operator via a push button.

Operator-Declared Rejects

The quality logic mimics real industrial behavior. The operator signals only the non-conforming parts and the system deduces the good count automatically. This avoids forcing the operator to validate every part, which would be impractical in a real line.

5 Results

The system was validated by running the full chain from the Wokwi simulation to the Node-RED dashboard. The ESP32 published a JSON message every two seconds, the OEE values updated live on the dashboard and the alert system triggered correctly when the OEE value crossed the critical threshold.

- The acquisition chain from sensors to MQTT ran without interruption.
- TD, TP, TQ and OEE updated every two seconds on the dashboard.
- The red LED and buzzer activated when OEE dropped below 45%.
- The Telegram and email notifications were sent to the production manager.

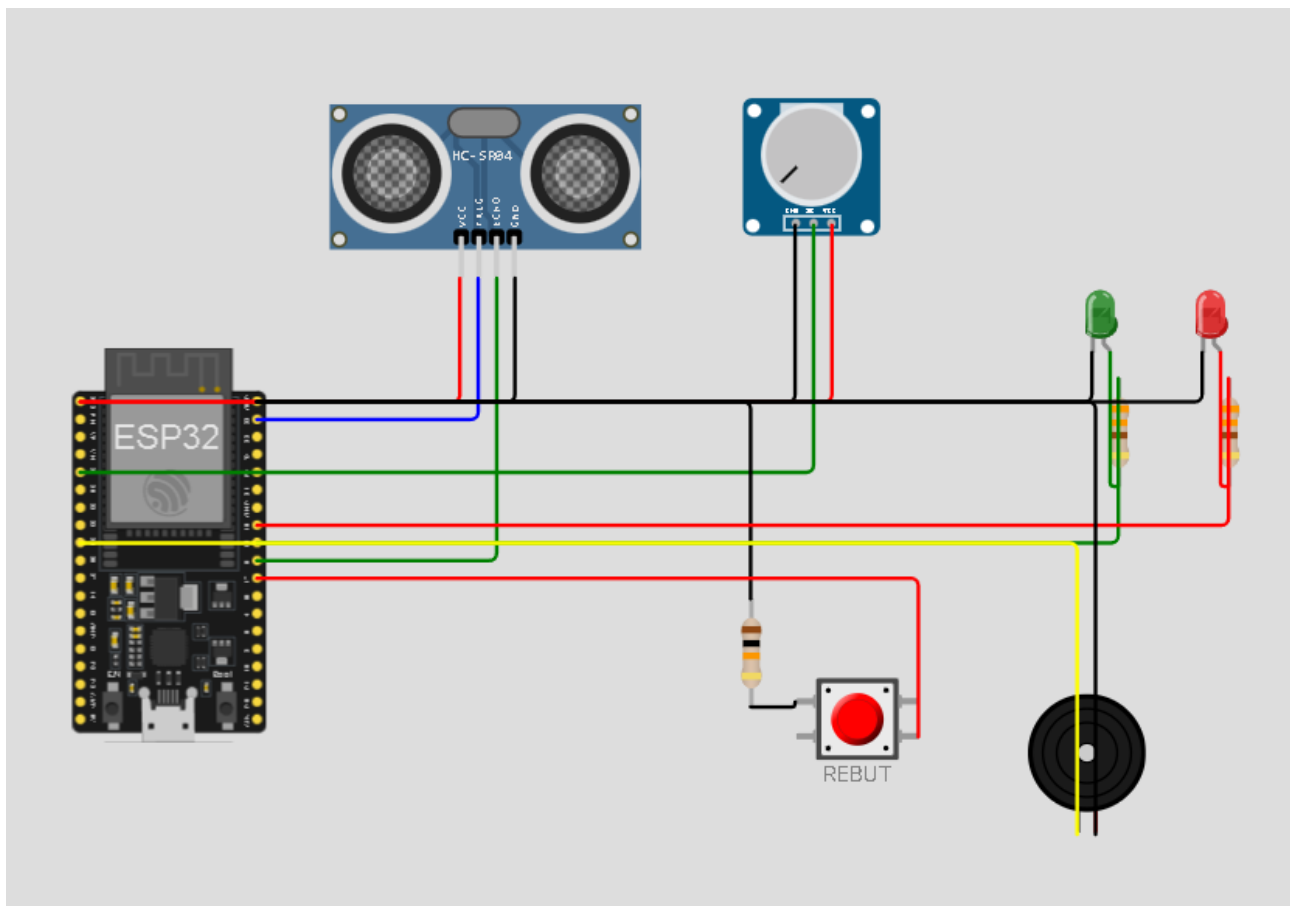


Figure 2: The ESP32 circuit simulated in Wokwi (potentiometer, ultrasonic sensor, button, LEDs, buzzer).

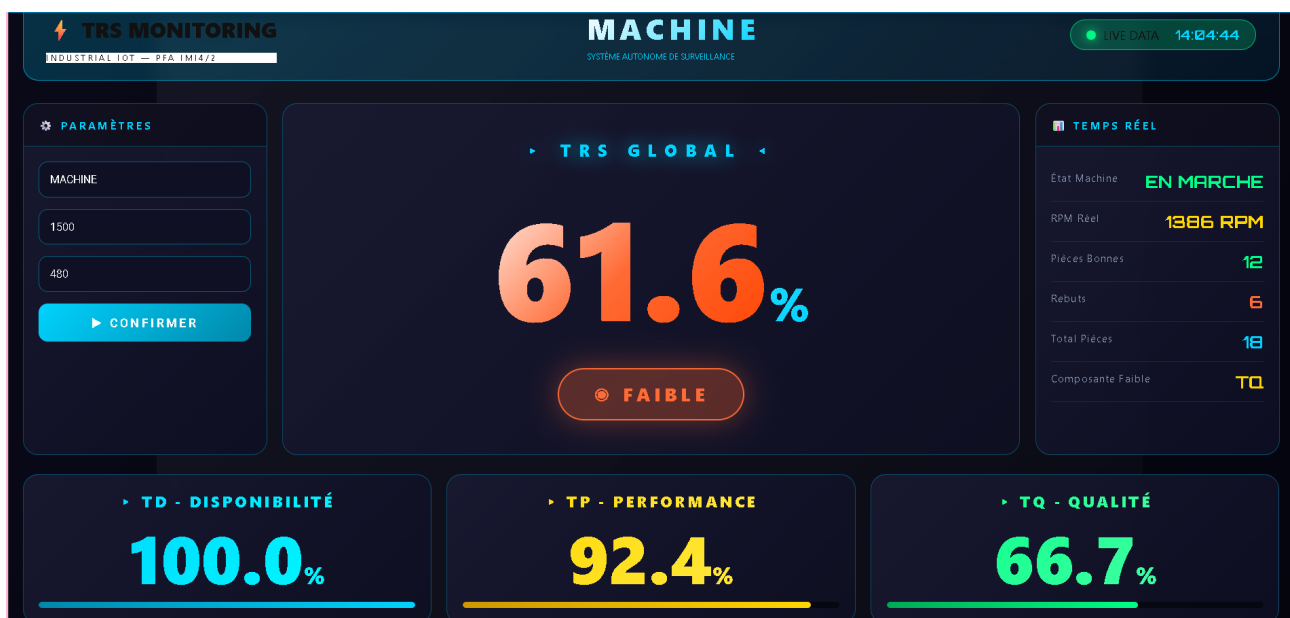


Figure 3: The Node-RED live dashboard showing OEE, TD, TP, TQ, RPM, machine state and the alert indicator.

6 Alert System

The system triggers automatic alerts based on the global OEE value. Four levels are defined, each mapped to a specific visual and audio response.

Table 1: Alert levels triggered by OEE thresholds.

OEE range	Level	Signal	Action
$\geq 85\%$	Excellent	Green LED steady	Maintain
$65\% - 85\%$	Average	No LED	Monitor
$45\% - 65\%$	Low	Red LED blinking	Attention
$< 45\%$	Critical	Red LED + buzzer (1000 Hz, 500 ms)	Immediate intervention

Two Alert Layers

The system provides two independent alert layers. The ESP32 firmware drives a local buzzer and LEDs for the operator standing in front of the machine. The Node-RED flow simultaneously sends email and Telegram notifications to the production manager. This separation ensures both local and remote awareness without overloading the machine controller.

The alerts are triggered every two seconds, synchronized with the MQTT publication interval. No manual intervention is required to reset the state. The system re-evaluates the level on every incoming message.

7 Industrial Perspective

The simulation validates the algorithms. The industrial deployment would replace the ESP32 and virtual sensors with a Raspberry Pi 4B and three non-invasive physical sensors.

Table 2: Physical sensors for the industrial deployment.

Component	Sensor	Measured quantity
Availability (TD)	Hall-effect current sensor SCT-013	Current (0–30 A)
Performance (TP)	MEMS accelerometer ADXL345 + FFT	Vibration (3 axes)
Quality (TQ)	USB camera + TensorFlow Lite	Image classification

Data would be forwarded to an ERP system through a REST middleware. The deployment requires sensors, a processing unit, an industrial enclosure, backup power and consumables. Labour, ERP licensing, middleware development and on-site commissioning are not included and depend on the target company.

8 Discussion

8.1 What Worked

- The full chain from simulated sensor to live dashboard runs end to end.
- OEE computation is consistent between the ESP32 firmware and the Node-RED re-computation.
- Alerts trigger correctly when OEE falls below the critical threshold.
- The dashboard updates every two seconds with live values and history.

8.2 Limitations

Important Limitations

- Simulation only. No physical sensors were used.
- OEE values depend on simulated machine behavior.
- Alerts are threshold-based, not predictive.
- Storage is local SQLite with no long-term analytics.
- Single machine scope. A full production line would require multiple nodes.

8.3 Lessons Learned

How MQTT Decouples Acquisition from Processing

The ESP32 does not know who consumes its data. The dashboard does not know who produces it. This decoupling allows the acquisition side and the processing side to evolve independently, which is essential in industrial settings where machines and software are maintained by different teams.

Why Operator-Declared Rejects Are Realistic

Requiring the operator to validate every good part is unrealistic. Real industrial systems trust the operator to signal only exceptions. This logic was implemented and validated in the simulation.

From Simulation to Deployment

Validating the algorithms in Wokwi before buying any hardware is a pragmatic approach. It proves that the OEE logic is correct before any industrial investment and reduces deployment risk.

9 Conclusion

This project demonstrates that a complete OEE monitoring system can be built with open-source tools and validated in simulation before any industrial deployment. The algorithm is correct, the alert system is functional, the architecture is sound and the path to real deployment is clear. Beyond OEE measurement, the same architecture opens the door to predictive maintenance, ERP integration and advanced production analytics.

10 Tools and Environment

Table 3: Tools, technologies and hardware used in this project.

Category	Tool or Component	Role in the project
Microcontroller	ESP32 DevKit C V4	Acquisition, local OEE computation, MQTT client, alert control
Sensor	Potentiometer 10 k Ω	Simulates current consumption and rotation speed
Sensor	Ultrasonic HC-SR04	Counts good parts by distance detection
Input	Push button (red)	Operator-declared rejects
Output	LED green and LED red	Visual status of parts and alerts
Output	Piezoelectric buzzer	Audio alert when OEE is critical
Simulator	Wokwi	Full circuit and firmware simulation in the browser
Firmware	Arduino C++	ESP32 code: Wi-Fi, MQTT, debounce, OEE logic
Protocol	MQTT	Publish and subscribe transport over Wi-Fi
Broker	broker.mqtttdashboard.co	Public MQTT broker for message routing
Flow engine	Node-RED	Parsing, OEE re-computation, alert logic, dashboard
Storage	SQLite	Local time-series storage of OEE records
Dashboard	node-red-dashboard	Live gauges, history graph, CSV and HTML export
Alerts	Email and Telegram nodes	Remote notification to the production manager

Oumaima Yaakoubi

Instrumentation and Intelligent Systems, INSAT, Tunisia

[linkedin.com/in/oumaima-yaakoubi](https://www.linkedin.com/in/oumaima-yaakoubi) • github.com/o-Yaakoubi